

UNIT-3

Computer Architecture Basics (3 Hours)

Contents

3.1. History of Computer Architecture

3.2. Overview of Computer Organization, Architecture of Basic Computer:
Register organization, Common Bus System, Instruction
Format, Register Stack and Memory Stack

Introduction to Computer Architecture

- Computer architecture is the conceptual design, structure, and operational framework of a computer system.
- Computer architecture is a specification detailing how a set of software and hardware technology standards interact to form a computer system or platform.
- In short, computer architecture refers to how a computer system is designed and what technologies it is compatible with.
- As with other contexts and meanings of the word architecture, computer architecture is likened to the art of determining the needs of the user/system/technology, and creating a logical design and standards based on those requirements.
- A very good example of computer architecture is von Neumann architecture, which is still used by most types of computers today.
- This was proposed by the mathematician John von Neumann in 1945.

There are three categories of computer architecture:

A. System Design:

- This includes all hardware components in the system, including data processors aside from the CPU, such as the graphics processing unit and direct memory access.
- It also includes memory controllers, data paths and miscellaneous things like multiprocessing and virtualization.

B. Instruction Set Architecture (ISA):

- This is the embedded programming language of the central processing unit.
- It defines the CPU's functions and capabilities based on what programming it can perform or process.
- This includes the word size, processor register types, memory addressing modes, data formats and the instruction set that programmers use.

C. Microarchitecture:

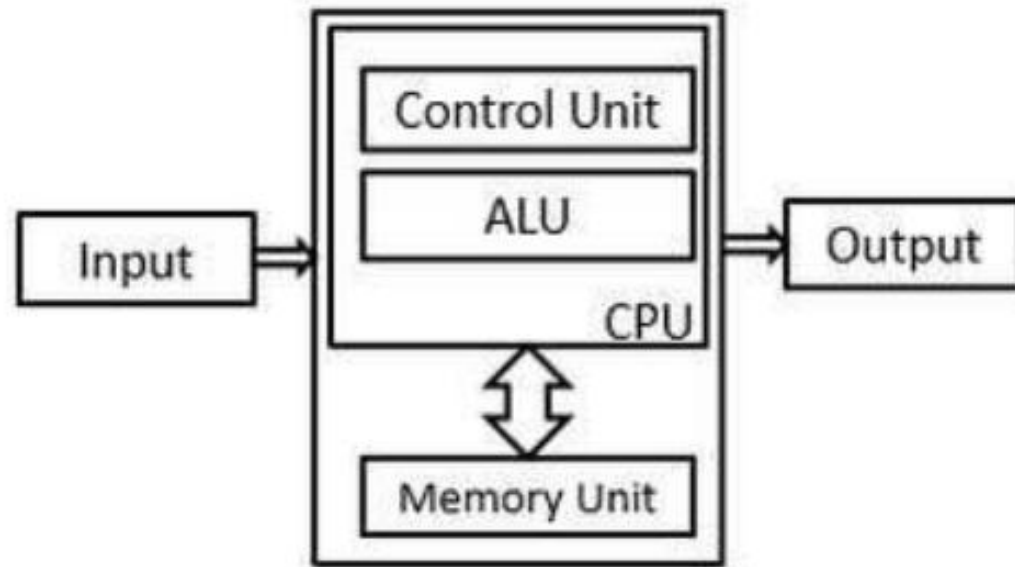
- Otherwise known as computer organization, this type of architecture defines the data paths, data processing and storage elements, as well as how they should be implemented in the ISA.

History of Computer Architecture

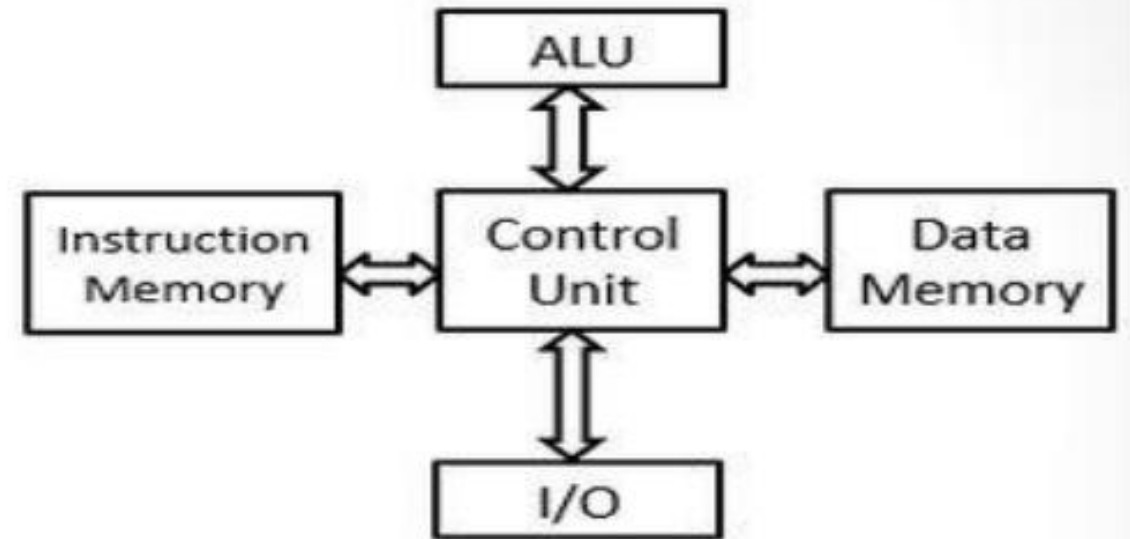
- The first document of Computer Architecture was a correspondence between Charles Babbage and Ada Lovelace, that describes the analytical engine.
- Here is the example of other early important machines: John Von Neumann and Alan Turing and; Harvard Architecture.
- Computer architecture is the art of determining the needs of the user of a structure and then designing to meet those needs as effectively as possible with economic status and as well as the technological constraints.
- In ancient period, computer architectures were designed and prepared on the paper and then, directly built into the final hardware form.
- Later, in today`s computer architecture, prototypes were physically built in the form of transistor logic (TTL) computer such as the prototypes of the 6800 and the PA-RISC tested, and tweaked before committing to the final hardware form.

Computer architecture has evolved through several generations:

- **1st Generation (1940s–1950s)**
 - ✓ Used vacuum tubes
 - ✓ ENIAC (1945) — first general-purpose electronic computer
 - ✓ Machine language programming only
 - ✓ Massive size, high power consumption
- **2nd Generation (1950s–1960s)**
 - ✓ Transistors replaced vacuum tubes
 - ✓ Assembly language introduced
 - ✓ IBM 7090, CDC 1604
- **3rd Generation (1960s–1970s)**
 - ✓ Integrated Circuits (ICs)
 - ✓ IBM System/360 — first family of compatible computers
 - ✓ High-level languages (FORTRAN, COBOL)
- **4th Generation (1970s–present)**
 - ✓ Microprocessors (VLSI technology)
 - ✓ Intel 4004 (1971) — first microprocessor
 - ✓ Personal computers emerged
- **5th Generation (present & beyond)**
 - ✓ Parallel processing, AI-based systems
 - ✓ Multi-core processors, quantum computing



Von Neumann Model



Harvard Model

Overview of Computer Organization

- **Computer organization** refers to how the physical hardware components of a computer system are interconnected and interact to execute instructions.
- It acts as the structural realization of **computer architecture**, translating abstract design—like instruction sets—into operational hardware.
- Computer organization refers to the operational units and their interconnection that realize the architecture specification.
- Computer organization deals with physical aspects of computer design, memory and their types and microprocessors design.
- Computer organization is concerned with the way the hardware components operate and the way they are connected together to form a computer system.
- It describes how the computer performs. Examples: circuit design, control signals, memory types and etc.

Computer Organization vs Architecture

1. Computer Architecture

- refers to those attributes of a system that have a direct impact on the logical execution of a program. Examples:
 - the instruction set
 - the number of bits used to represent various data types
 - I/O mechanisms
 - memory addressing techniques

Computer Organization

- refers to the operational units and their interconnections that realize the architectural specifications. Examples are things that are transparent to the programmer:
 - control signals
 - interfaces between computer and peripherals
 - the memory technology being used.

2. So, for example, the fact that a multiply instruction is available is a computer architecture issue. How that multiply is implemented is a computer organization issue.

3. Architecture

- is those attributes visible to the programmer
 - Instruction set, number of bits used for data representation, I/O mechanisms, addressing techniques. e.g. Is there a multiply instruction?

Organization

- is how features are implemented
 - Control signals, interfaces, memory technology. e.g. Is there a hardware multiply unit or is it done by repeated addition?

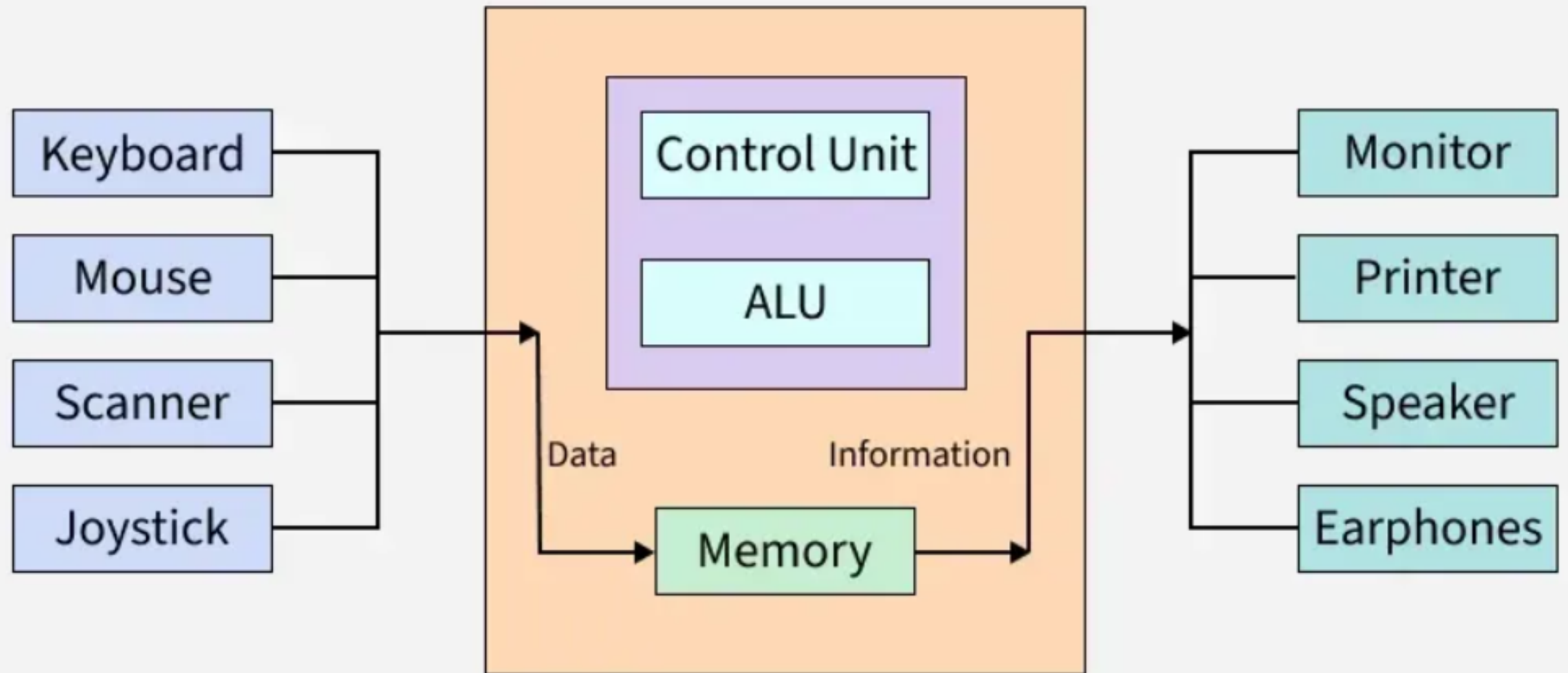
4. Computer architecture

- is concerned with the structure and behavior of computer system as seen by the user.

Computer organization

- is concerned with the way the hardware components operate and the way they are connected together to form a computer system.

Basic Architecture of a Computer



Register

- **Register:** A register is a small, high-speed storage location within the CPU (Central Processing Unit) of a computer that is used to store temporary data and instructions that are actively being processed by the CPU.

A register may hold an instruction, a storage address, or any kind of data (such as a bit sequence or individual characters). Some instructions specify registers as part of the instruction. For example, an instruction may specify that the contents of two defined registers be added together and then placed in a specified register.

Registers are the most important components of CPU. Each register performs a specific function. A brief description of most important CPU's registers and their functions are given below:

1. **Memory Address Register (MAR):** This register holds the address of memory where CPU wants to read or write data. When CPU wants to store some data in the memory or reads the data from the memory, it places the address of the required memory location in the MAR.

2. **Memory Buffer Register (MBR):** This register holds the contents of data or instruction read from, or written in memory. The contents of instruction placed in this register are transferred to the Instruction Register, while the contents of data are transferred to the accumulator or I/O register. In other words you can say that this register is used to store data/instruction coming from the memory or going to the memory.
3. **Program Counter (PC):** Program Counter register is also known as Instruction Pointer Register. This register is used to store the address of the next instruction to be fetched for execution. When the instruction is fetched, the value of IP is incremented. Thus this register always points or holds the address of next instruction to be fetched.
4. **Instruction Register (IR):** Once an instruction is fetched from main memory, it is stored in the Instruction Register. The control unit takes instruction from this register, decodes and executes it by sending signals to the appropriate component of computer to carry out the task.

5. **Accumulator Register:** The accumulator register is located inside the ALU; it is used during arithmetic & logical operations of ALU. The control unit stores data values fetched from main memory in the accumulator for arithmetic or logical operation. This register holds the initial data to be operated upon, the intermediate results, and the final result of operation. The final result is transferred to main memory through MBR.

6. **Stack Control Register:** A stack represents a set of memory blocks; the data is stored in and retrieved from these blocks in an order, i.e. First In and Last Out (FILO). The Stack Control Register is used to manage the stacks in memory. The size of this register is 2 or 4 bytes.

7. **Flag Register:** The Flag register is used to indicate occurrence of a certain condition during an operation of the CPU. It is a special purpose register with size one byte or two bytes. Each bit of the flag register constitutes a flag (or alarm), such that the bit value indicates if a specified condition was encountered while executing an instruction.

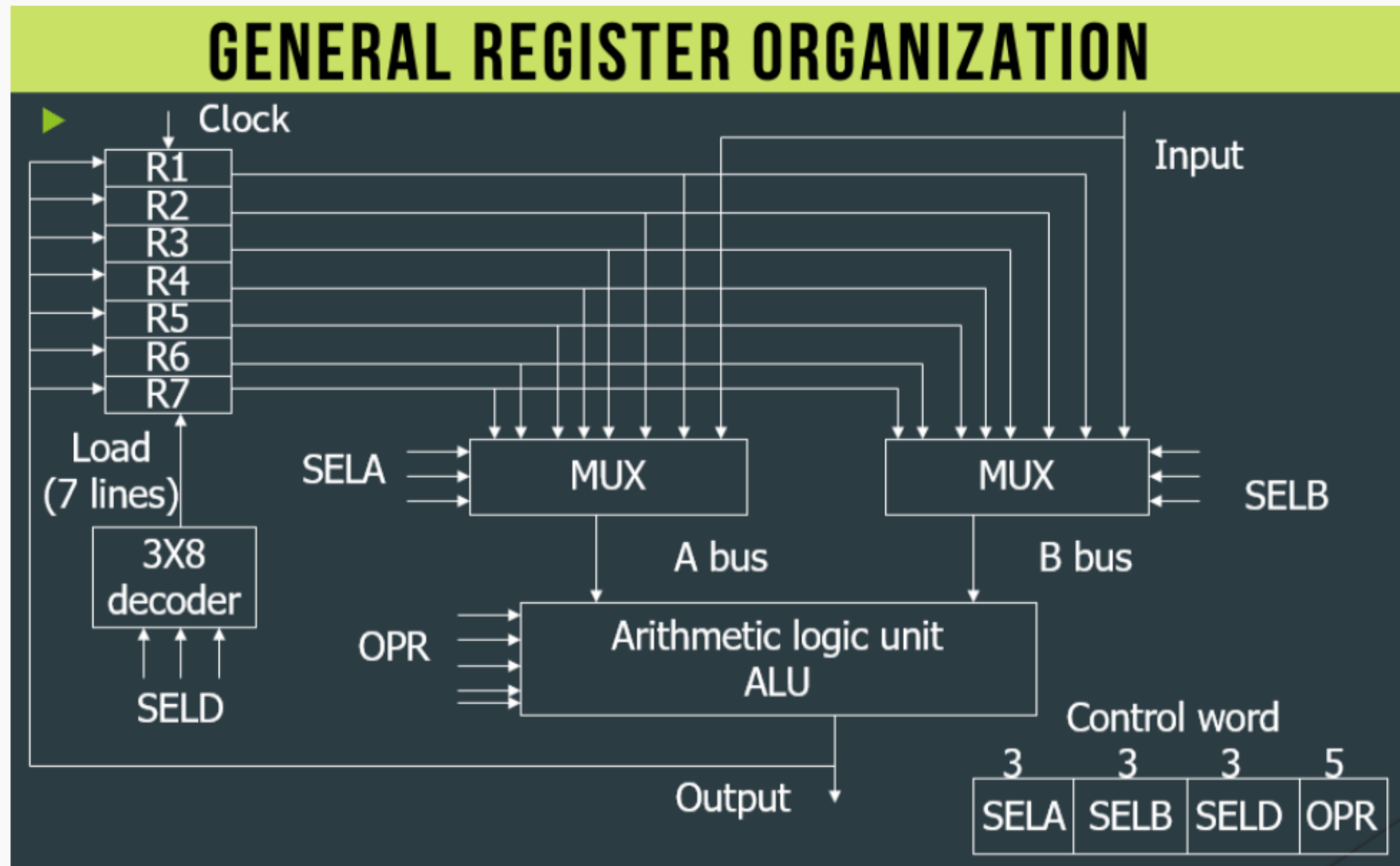
Register Organization

- **Register organization** refers to the way registers are structured and managed within the CPU.
- This includes the arrangement of different types of registers, such as data registers, address registers, and control registers.
- **Register organization** in computer architecture defines the physical structuring, quantifying, and functional grouping of registers within a Central Processing Unit (CPU).
- Efficient register organization is crucial for optimizing the performance of the CPU and ensuring smooth execution of instructions.

General Register Organization

- **General register organization** refers to the arrangement and management of various types of registers within the Central Processing Unit (CPU) of a computer system.
- These registers play essential roles in the execution of instructions and the manipulation of data within the CPU.
- These registers are used for various computational and data manipulation tasks during program execution.
- The organization typically defines the number of registers available, their size, and their specific roles, including temporary data storage, addressing, and operand manipulation.
- The organization of general registers can vary between different CPU architectures and designs, impacting the CPU's performance and capabilities.

Block Diagram of General Register Organization (7 Registers)



- A decoder is used to select a particular register. The output of each register is connected to two multiplexers to form the two buses A and B. The selection lines in each multiplexer select the input data for the particular bus.
- The A and B buses form the two inputs of an ALU. The operation select lines decide the micro operation to be performed by ALU.
- The result of the micro operation is available at the output bus. The output bus connected to the inputs of all registers, thus by selecting a destination register it is possible to store the result in it.

The control unit that operates the CPU bus system directs the information flow through the registers and ALU by selecting the various components in the system. For example, to perform the operation

$$R1 \leftarrow R2 + R3$$

the control must provide binary selection variables to the following selector inputs:

1. MUX A selector (SELA): to place the content of R2 into bus A.
2. MUX B selector (SELB): to place the content of R3 into bus B.
3. ALU operation selector (OPR): to provide the arithmetic addition $A + B$.
4. Decoder destination selector (SELD): to transfer the content of the output bus into R 1.

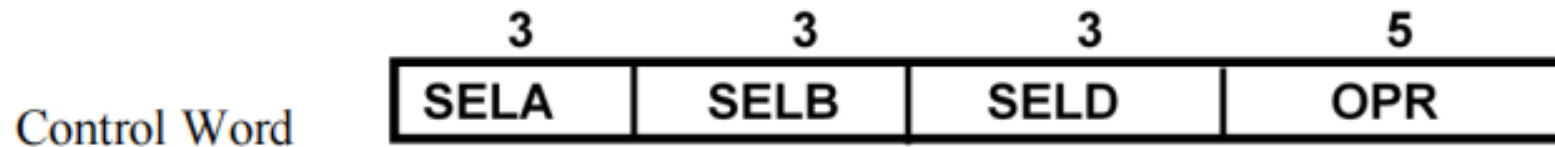
Control Word

There are 14 binary selection inputs in the unit, and their combined value specifies a control word.



FORMATE OF CONTROL WORD

1. The three bit of SELA select a source registers of the **a** input of the ALU.
2. The three bits of SELB select a source registers of the **b** input of the ALU.
3. The three bits of SELED or SELREG select a destination register using the decoder.
4. The ~~four~~ ^{five} bits of SELOPR select the operation to be performed by ALU.



Encoding of register selection fields

Binary Code	SELA	SELB	SELD
000	Input	Input	None
001	R1	R1	R1
010	R2	R2	R2
011	R3	R3	R3
100	R4	R4	R4
101	R5	R5	R5
110	R6	R6	R6
111	R7	R7	R7

Encoding of ALU operations

OPR		
Select	Operation	Symbol
00000	Transfer A	TSFA
00001	Increment A	INCA
00010	ADD A + B	ADD
00101	Subtract A - B	SUB
00110	Decrement A	DECA
01000	AND A and B	AND
01010	OR A and B	OR
01100	XOR A and B	XOR
01110	Complement A	COMA
10000	Shift right A	SHRA
11000	Shift left A	SHLA

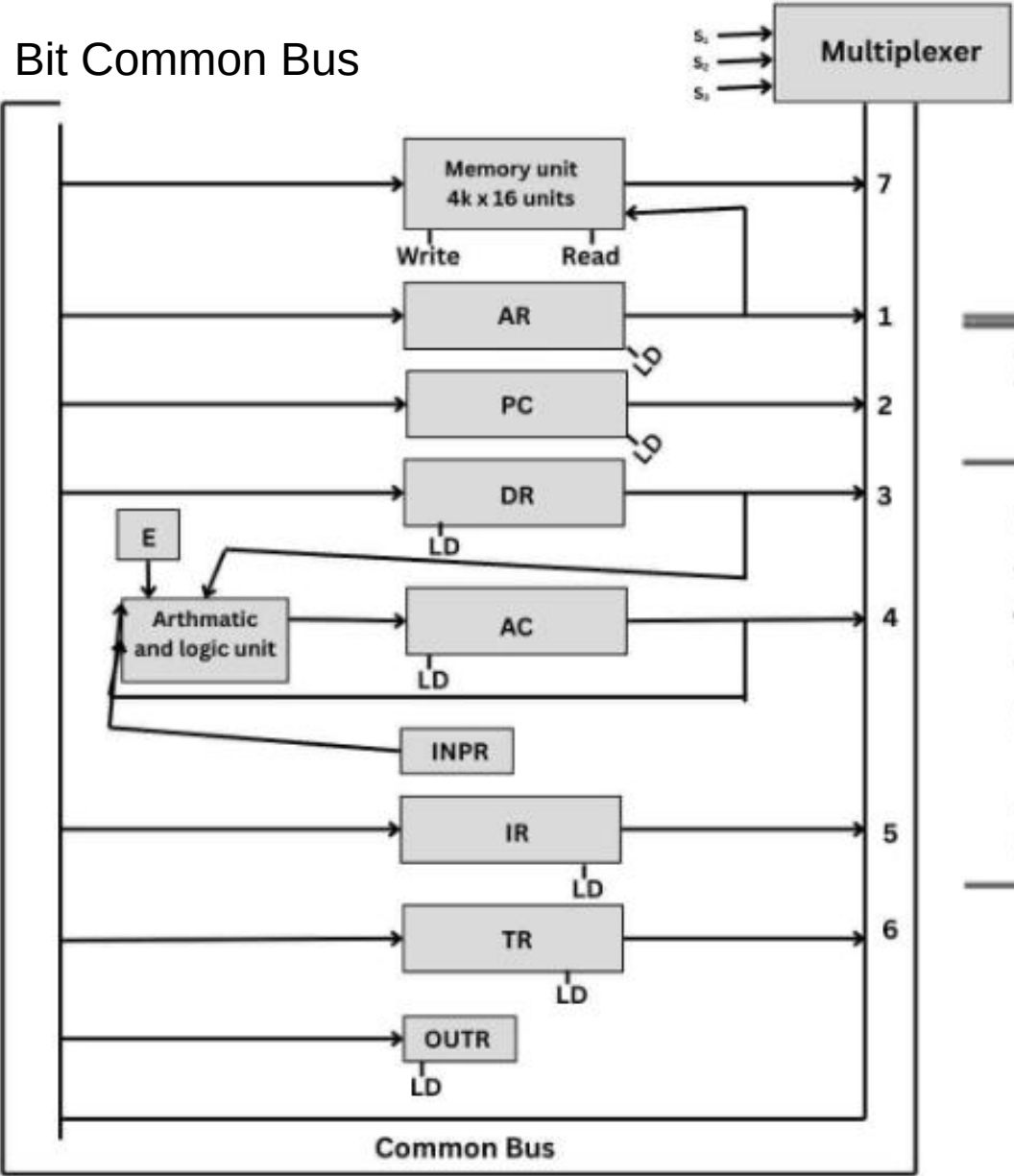
Examples of ALU Microoperations

Symbolic Designation					Control Word
Microoperation	SELA	SELB	SELD	OPR	
$R1 \leftarrow R2 - R3$	R2	R3	R1	SUB	010 011 001 00101
$R4 \leftarrow R4 \vee R5$	R4	R5	R4	OR	100 101 100 01010
$R6 \leftarrow R6 + 1$	R6	-	R6	INCA	110 000 110 00001
$R7 \leftarrow R1$	R1	-	R7	TSFA	001 000 111 00000
$\text{Output} \leftarrow R2$	R2	-	None	TSFA	010 000 000 00000
$\text{Output} \leftarrow \text{Input}$	Input	-	None	TSFA	000 000 000 00000
$R4 \leftarrow \text{shl } R4$	R4	-	R4	SHLA	100 000 100 11000
$R5 \leftarrow 0$	R5	R5	R5	XOR	101 101 101 01100

Common Bus System

- The basic computer system has various registers, a memory unit, and a control unit.
- The information is transferred from one unit to another to complete the required operation.
- Therefore, a path must be provided between these units.
- If separate lines are used to provide the path, the number of wires will be excessive in the system.
- A more efficient method for transferring information between different units of a computer system is a common bus system.
- A common bus can be constructed using a tri-state buffer or a multiplexer.
- A common bus system is used to transfer the information between different units of the computer system.

16 Bit Common Bus



Register symbol	Number of bits	Register name	Function
<i>DR</i>	16	Data register	Holds memory operand
<i>AR</i>	12	Address register	Holds address for memory
<i>AC</i>	16	Accumulator	Processor register
<i>IR</i>	16	Instruction register	Holds instruction code
<i>PC</i>	12	Program counter	Holds address of instruction
<i>TR</i>	16	Temporary register	Holds temporary data
<i>INPR</i>	8	Input register	Holds input character
<i>OUTR</i>	8	Output register	Holds output character

Common Bus System

Working Principle of Common Bus System

- The outputs of registers and memory units are connected to the common bus.
- In the same manner, the common bus is also connected to the inputs of each register and the data inputs of the memory.
- Whenever a register wants to receive data from a common bus, one of the control signal load (LD) must be enabled.
- The particular register which is having load (LD) control signal enabled receives the data from the common bus during the next clock pulse.
- If two or more registers have the LD control signal enabled simultaneously, they may receive the same data from the common bus.
- The memory receives the contents of the common bus only when its write input is enabled.
- The multiplexer is used to select a specific output for the common bus at any given time.
- There are three selection lines for the multiplexer.

- The binary values in these lines activate the specific output from the registers.
- The number along each output shows the decimal equivalent of the required binary selection.
- For example, the accumulator (AC) can place the output on the common bus if the selection lines contain 100 (4).
- As shown in the above figure, the input data and output data of the memory unit are connected to the common bus, and the memory can perform a reading or writing operation when the proper control signal (read or write).
- However, the memory address is connected to AR only; this register must be used to specify the address of a register in the memory unit where the reading or writing operation is to be performed.
- This kind of bus architecture eliminates the requirement of a separate address bus.
- The memory unit can receive the data from any of the registers, and similarly, all the registers except the accumulator (AC) can receive the data from the memory unit directly.
- In a common bus system, there is an arithmetic and logic unit.
- It has three inputs.
- One set of 16-bit input lines came from the output lines of the accumulator.

- This set can be used to perform the micro operations with the accumulator only, like shift AC and complement AC.
- Another set of 16-bit input lines comes from the data register (DR).
- This set is used to perform micro operations that have two operands, like addition.
- In such a case, one operand is stored in the accumulator and the other operand in the data register.
- A third set of 8-bit inputs comes from the input register (INPR).
- This set is used to communicate with the input device.
- The input register (INPR) and output register (OUTR) are used to communicate with the I/O devices.
- The serial information from the input device is stored in the input register.
- It converts this serial information into parallel and transfers it to the arithmetic and logic unit.
- Similarly, the serial information for the output device is stored in the output register.
- The output register transfers the parallel information in a serial manner to the output device.
- These two registers communicate with I/O devices serially and with the accumulator in parallel.

Stack Organization

- A useful feature that is included in the CPU of most computers is a stack or last-in, first-out (LIFO) list.
- A stack is a storage device that stores information in such a manner that the item stored last is the first item retrieved.
- The operation of a stack can be compared to a stack of trays. The last tray placed on top of the stack is the first to be taken off.
- The register that holds the address for the stack is called a stack pointer (SP) because its value always points at the top item in the stack.
- The two operations of a stack are the insertion and deletion of items. The operation of insertion is called push (or push-down) because it can be thought of as the result of pushing a new item on top. The operation of deletion is called pop (or pop-up) because it can be thought of as the result of removing one item so that the stack pops up. However, nothing is pushed or popped in a computer stack. These operations are simulated by incrementing or decrementing the stack pointer register.

Stack Operations

The two operations of a stack are:

1. **Push:** Inserts an item on top of stack.
2. **Pop:** Deletes an item from top of stack.

Implementation of Stack

In digital computers, stack can be implemented in two ways:

1. Register Stack
2. Memory Stack

Register Stack

A stack can be placed in a portion of a large memory or it can be organized as a collection of a finite number of memory words or registers. Figure 8-3 shows the organization of a 64-word register stack.

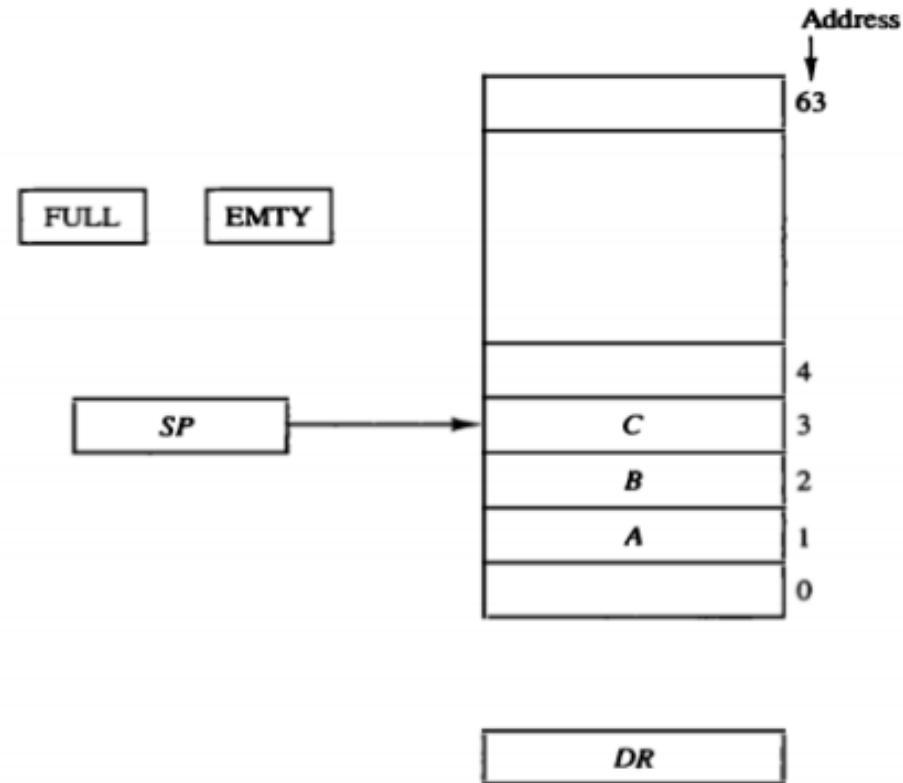


Figure 8-3 Block diagram of a 64-word stack.

The stack pointer register SP contains a binary number whose value is equal to the address of the word that is currently on top of the stack. Three items are placed in the stack: A, B, and C, in that order. Item C is on top of the stack so that the content of SP is now 3.

To remove the top item, the stack is popped by reading the memory word at address 3 and decrementing the content of SP. Item B is now on top of the stack since SP holds address 2.

To insert a new item, the stack is pushed by incrementing SP and writing a word in the next-higher location in the stack.

Initially, SP is cleared to 0, EMTY is set to 1, and FULL is cleared to 0, so that SP points to the word at address 0 and the stack is marked empty and not full. If the stack is not full (if $FULL = 0$), a new item is inserted with a push operation. The push operation is implemented with the following sequence of micro operations;

$SP \leftarrow SP + 1$	Increment stack pointer
$M[SP] \leftarrow DR$	Write item on top of the stack

A new item is deleted from the stack if the stack is not empty (if $EMTY = 0$). The pop operation consists of the following sequence of micro operations:

$DR \leftarrow M[SP]$	Read item from the top of stack
$SP \leftarrow SP - 1$	Decrement stack pointer

Memory Stack

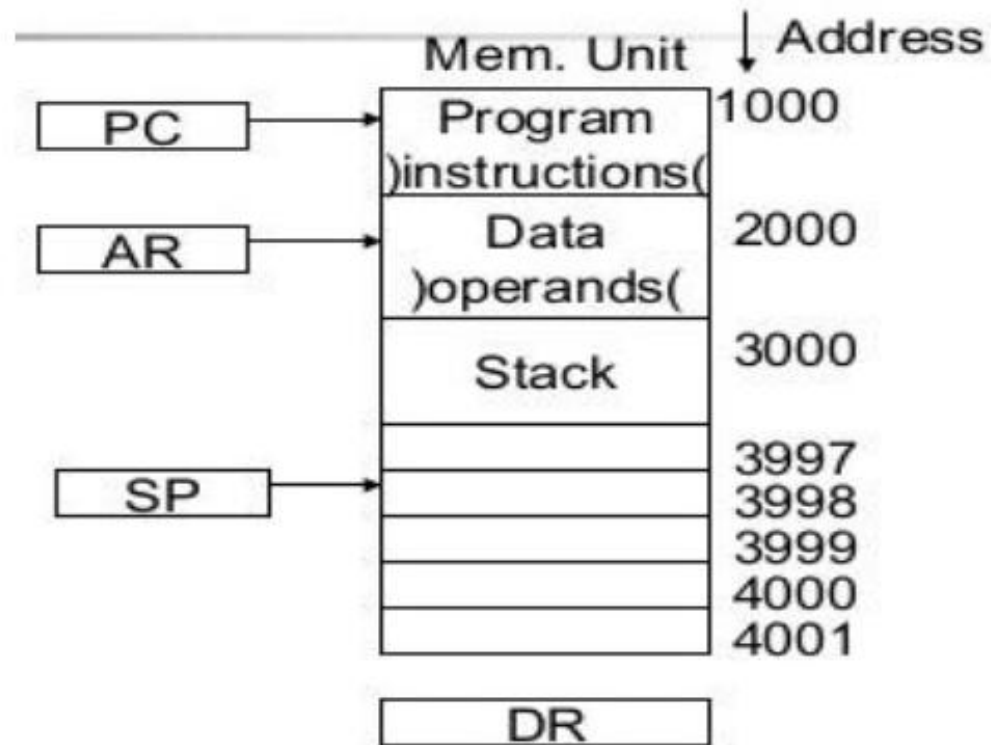
A stack can be implemented in a random-access memory attached to a CPU.

The implementation of a stack in the CPU is done by assigning a portion of memory to a stack operation and using a processor register as a stack pointer.

The portion of computer memory is partitioned into three segments: program, data, and stack.

The program counter PC points at the address of the next instruction in the program. The address register AR points at an array of data. The stack pointer SP points at the top of the stack.

PC is used during the fetch phase to read an instruction. AR is used during the execute phase to read an operand. SP is used to push or pop items into or from the stack.



Computer memory with
program, data & stack segments

As shown in Fig., the initial value of SP is 4001 and the stack grows with decreasing addresses. Thus the first item stored in the stack is at address 4000 the second item is stored at address 3999 and the last address that can be used for the stack is 3000.

We assume that the items in the stack communicate with a data register DR. A new item is inserted with the push operation as follows:

$SP \leftarrow SP - 1$

$M[SP] \leftarrow DR$

The stack pointer is decremented so that it points at the address of the next word. A memory write operation inserts the word from DR into the top of the stack.

A new item is deleted with a pop operation as follows:

$DR \leftarrow M[SP]$

$SP \leftarrow SP + 1$

The top item is read from the stack into DR. The stack pointer is then incremented to point at the next item in the stack.

Instruction Format

Computer perform task on the basis of instruction provided. **An instruction format** defines layout of bits of an instruction.

An instruction in computer comprises of groups called fields. These field contains different information as for computers every thing is in 0 and 1 so each field has different significance on the basis of which a CPU decide what so perform.

The most common fields are:

- Operation field which specifies the operation to be performed like addition.
- Address field which contain the location of operand, i.e., register or memory location.
- Mode field which specifies how operand is to be founded.

An instruction is of various length depending upon the number of addresses it contain.

Generally, CPU organization are of three types on the basis of number of address fields:

- Single Accumulator organization
- General register organization
- Stack organization

In **first** organization operation is done involving a special register called accumulator. In **second** on multiple registers are used for the computation purpose. In **third** organization the work on stack basis operation due to which it does not contain any address field.

1. Single Accumulator organization

All operations are performed with an accumulator register. The instruction format in this type of computer uses one address field. For example, the instruction that specifies an arithmetic addition is defined by an assembly language instruction as:

ADD X

where X is the address of the operand. The ADD instruction in this case results in the operation $AC \leftarrow AC + M[X]$. AC is the accumulator register and M[X] symbolizes the memory word located at address X.

2. General register organization

The instruction format in this type of computer needs three register address fields. Thus the instruction for an arithmetic addition may be written in an assembly language as:

ADD R1, R2, R3

to denote the operation $R1 \leftarrow R2 + R3$.

The number of address fields in the instruction can be reduced from three to two if the destination register is the same as one of the source registers. Thus the instruction

ADD R1, R2

would denote the operation $R1 \leftarrow R1 + R2$.

3. Stack organization

Computers with stack organization would have PUSH and POP instructions which require an address field. Thus the instruction

PUSH X

will push the word at address X to the top of the stack. The stack pointer is updated automatically.

Operation-type instructions do not need an address field in stack-organized computers. This is because the operation is performed on the two items that are on top of the stack. The instruction

ADD

in a stack computer consists of an operation code only with no address field. This operation has the effect of popping the two top numbers from the stack, adding the numbers, and pushing the sum into the stack. There is no need to specify operands with an address field since all operands are implied to be in the stack.

Instruction Format

To illustrate the influence of the number of addresses on computer programs, we will evaluate the arithmetic statement

$$X = (A + B) * (C + D)$$

using zero, one, two, or three address instructions. We will use the symbols **ADD**, **SUB**, **MUL**, and **DIV** for the four arithmetic operations; **MOV** for the transfer-type operation; and **LOAD** and **STORE** for transfers to and from memory and AC register. We will assume that the operands are in memory addresses A, B, C, and D, and the result must be stored in memory at address X.

1. Three Address Instructions

Computers with three-address instruction formats can use each address field to specify either a processor register or a memory operand.

The program in assembly language that evaluates $X = (A + B) * (C + D)$ is shown below, together with comments that explain the register transfer operation of each instruction.

ADD	R1, A, B	$R1 \leftarrow M[A] + M[B]$
ADD	R2, C, D	$R2 \leftarrow M[C] + M[D]$
MUL	X, R1, R2	$M[X] \leftarrow R1 * R2$

It is assumed that the computer has two processor registers, R1 and R2. The symbol M[A] denotes the operand at memory address symbolized by A.

The advantage of the three-address format is that it results in short programs when evaluating arithmetic expressions. The disadvantage is that the binary-coded instructions require too many bits to specify three addresses.

2. Two-Address Instructions

Two-address instructions are the most common in commercial computers. Here again each address field can specify either a processor register or a memory word. The program to evaluate $X = (A + B) * (C + D)$ is as follows:

MOV	R1, A	$R1 \leftarrow M[A]$
ADD	R1, B	$R1 \leftarrow R1 + M[B]$
MOV	R2, C	$R2 \leftarrow M[C]$
ADD	R2, D	$R2 \leftarrow R2 + M[D]$
MUL	R1, R2	$R1 \leftarrow R1 * R2$
MOV	X, R1	$M[X] \leftarrow R1$

The **MOV** instruction moves or transfers the operands to and from memory and processor registers. The first symbol listed in an instruction is assumed to be both a source and the destination where the result of the operation is transferred.

3. One-Address Instructions

One-address instructions use an accumulator (AC) register for all data manipulation. For multiplication and division there is a need for a second register. However, here we will neglect the second register and assume that the AC contains the result of all operations. The program to evaluate $X = (A + B) * (C + D)$ is

LOAD	A	AC ← M[A]
ADD	B	AC ← AC + M[B]
STORE	T	M[T] ← AC
LOAD	C	AC ← M[C]
ADD	D	AC ← AC + M[D]
MUL	T	AC ← AC * M[T]
STORE	X	M[X] ← AC

All operations are done between the AC register and a memory operand. T is the address of a temporary memory location required for storing the intermediate result.

4. Zero-Address Instructions

A stack-organized computer does not use an address field for the instructions **ADD** and **MUL**. The **PUSH** and **POP** instructions, however, need an address field to specify the operand that communicates with the stack.

The following program shows how $X = (A + B) * (C + D)$ will be written for a stack organized computer. (**TOS** stands for top of stack.)

PUSH	A	TOS ← A
PUSH	B	TOS ← B
ADD		TOS ← (A + B)
PUSH	C	TOS ← C
PUSH	D	TOS ← D
ADD		TOS ← (C + D)
MUL		TOS ← (C + D) * (A + B)
POP	X	M[X] ← TOS

End of Chapter Three !